



축구 경기 영상 내의 다양한 객체 이미지 분할

※ 상태	완료
📅 마감일	@12/01/2025
≡ 설명	스프린트 미션 #8

스프린트 미션 #8

축구 경기 영상 내의 다양한 객체 이미지 분할

- 이번 미션에서는 U-Net을 이용해 축구 경기 영상 내의 다양한 객체(예: 골대, 심판, 선수, 관중 등)를 픽셀 단위로 분할하는 Semantic Segmentation 작업을 수행합니다. 실제 경기 장면에서 추출한 이미지를 바탕으로, 각 픽셀이 어떤 클래스에 속하는지 예측하는 모델을 직접 설계하고 학습해보세요.
- Cross Entropy Loss, Dice Loss 등 Segmentation 분야에서 사용하는 평가 지표를 활용해 모델 성능을 평가해 보세요.
- 사용 데이터셋 : 데이터 링크: **Football (Semantic Segmentation)**
 - 본 데이터셋은 **UEFA 슈퍼컵 2017** 레알 마드리드 vs 맨체스터 유나이티드 경기 하이라이트 영상을 기반으로 수집된 축구 경기 장면들로 구성되어 있습니다. 총 100장의 프레임을 사용하였으며, 매 12번째 프레임을 추출해 흐릿한 장면이나 이상치는 일부 대체했습니다.
 - **11개 클래스**가 포함되어 있습니다.
 - Goal Bar (골대)
 - Referee (심판)
 - Advertisement (광고판)
 - Ground (잔디)
 - Ball (축구공)

- Coaches & Officials (코칭 스태프 및 심판진)
- Audience (관중)
- Goalkeeper A (팀 A 골키퍼)
- Goalkeeper B (팀 B 골키퍼)
- Team A (팀 A 선수)
- Team B (팀 B 선수)

데이터 확인 및 전처리

1. 데이터 확인

1-1) 데이터 로드 / 경로 설정

```
# 이미지 파일 목록 정의
path = "football"
img_folder = os.path.join(path, "images")
img_list = os.listdir(img_folder)

# 원본이미지 / 마스크 이미지 찾기
image_files = sorted([name for name in img_list if name.endswith(".jpg")])
mask_files = sorted([name for name in img_list if "fuse" in name])

# 이미지 순서쌍 확인
image_pairs = []
for orig_img in image_files:
    # 동일한 프레임의 fuse 파일
    mask_img = next((name for name in mask_files if orig_img in name), None)
    image_pairs.append((orig_img, mask_img))

print(f"{len(image_pairs)} pairs")
```

2. 데이터 전처리

2-1) 마스크 색상 함수 정의

- 마스크 이미지로부터 색상 추출하는 함수 정의
색상 추출 후 라벨을 부여하여 모델의 타겟값으로 설정

```
# 데이터셋 전체의 고유한 색상 수집
def get_unique_colors(img_folder, mask_files, max_classes=11):
    color_set = set()

    for mask_file in mask_files:
        mask_path = os.path.join(img_folder, mask_file)
        mask_pil = Image.open(mask_path).convert("RGB") # 마스크 이미지를 불러오고
        mask = np.array(mask_pil)                        # np 배열로 바꿔준다음
        mask = mask.reshape(-1,3)                        # [픽셀수, 3] ⇒ RGB배열은 그
        # 대로 두고 픽셀을 아래로 펼쳐라
        unique_colors = np.unique(mask, axis=0)          # 모든 행들을 보고 중복값
        # 제거 후 나열

        for color in unique_colors:
            color_set.add(tuple(color)) # 고유한 색상 저장

        # 클래스 개수가 max_classes개가 되면 중단
        if len(color_set) >= max_classes:
            return list(color_set) # 최대 max_classes개까지만 반환

    return list(color_set) # 모든 마스크를 순회 후 반환

# 모든 마스크에서 등장하는 색상 수집
unique_colors = get_unique_colors(img_folder, mask_files)
color_to_label = {color: idx for idx, color in enumerate(unique_colors)}

def mask_rgb_to_label(mask, color_to_label):
    h, w, _ = mask.shape # 인자를 np.array 로 받음
    label_mask = np.zeros((h, w), dtype=np.int64)

    for color, label in color_to_label.items():
```

```

label_mask[(mask == color).all(axis=-1)] = label

return label_mask

```

2-2) 데이터셋 정의

```

class FootballDataset(Dataset):
    def __init__(self, image_files, mask_files, image_folder, color_to_label):
        self.image_files = image_files
        self.mask_files = mask_files
        self.image_folder = image_folder
        self.color_to_label = color_to_label
        self.img_transform = v2.Compose([
            v2.Resize((256, 256)),
            v2.ToImage(),
            v2.ToDtype(torch.float32, scale=True)
        ])

    def __len__(self):
        return len(self.image_files)

    def __getitem__(self, idx):
        img_path = os.path.join(self.image_folder, self.image_files[idx])
        mask_path = os.path.join(self.image_folder, self.mask_files[idx])

        img = Image.open(img_path).convert("RGB") # 원본 이미지 로드
        img = self.img_transform(img)

        mask = Image.open(mask_path).convert("RGB") # 마스크 이미지 로드
        mask = np.array(mask.resize((256, 256), resample=Image.Resampling.NEAREST))
        mask = mask_rgb_to_label(mask, self.color_to_label)
        mask = torch.from_numpy(mask).long()
        # (H, W) CrossEntropy 손실 함수에서 타겟 변수는 정수인덱스여야 함. 그래서
        # 정수로 dtype=torch.long

        return img, mask

```

2-3) 데이터 로더 정의

```
# 데이터 로드
dataset = FootballDataset(image_files, mask_files, img_folder, color_to_label)
train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size
train_dataset, test_dataset = torch.utils.data.random_split(
    dataset, [train_size, test_size])

train_loader = DataLoader(
    train_dataset, batch_size=2, shuffle=True, drop_last=False)

test_loader = DataLoader(
    test_dataset, batch_size=1, shuffle=False, drop_last=False)

print(f"Train Data: {len(train_dataset)}, Test Data: {len(test_dataset)}")
```

Semantic Segmentation 모델 UNet

1. UNet모델 정의

```
class DoubleConv(nn.Module): # (convolution ⇒ [BN] ⇒ ReLU) * 2
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
```

```

        nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(inplace=True),
    )

def forward(self, x):
    return self.block(x)

class Unet(nn.Module):
    def __init__(self, num_classes=11):
        super().__init__()
        self.encoder1 = DoubleConv(3, 64)
        self.encoder2 = DoubleConv(64, 128)
        self.encoder3 = DoubleConv(128, 256)

        self.bottleneck = DoubleConv(256, 512)

        self.pool = nn.MaxPool2d(2)

        self.upsample3 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride
=2)
        self.decoder3 = DoubleConv(512, 256)
        self.upsample2 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride
=2)
        self.decoder2 = DoubleConv(256, 128)
        self.upsample1 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=
2)
        self.decoder1 = DoubleConv(128, 64)

        self.output = nn.Conv2d(64, num_classes, kernel_size=1)

    def forward(self, x):
        e1 = self.encoder1(x)
        e2 = self.encoder2(self.pool(e1))
        e3 = self.encoder3(self.pool(e2))

        b = self.bottleneck(self.pool(e3))

```

```

d3 = self.upsample3(b)
d3 = torch.cat((d3, e3), dim=1)
# skip_connection ⇒ 디코더 단계에 맞는 인코더 출력물을 concat으로 붙임
d3 = self.decoder3(d3)

d2 = self.upsample2(d3)
d2 = torch.cat((d2, e2), dim=1)
d2 = self.decoder2(d2)

d1 = self.upsample1(d2)
d1 = torch.cat((d1, e1), dim=1)
d1 = self.decoder1(d1)

return self.output(d1)

```

2. 모델 학습 및 검증

2-1) 모델 학습

```

model = Unet().to(device)
loss_fn = nn.CrossEntropyLoss() # 손실함수 -> 다중분류에 적합한 CrossEntropy
optimizer = torch.optim.Adam(model.parameters(), lr=0.0005)

epochs = 50

for epoch in range(epochs):
    total_loss_UNet = 0

    model.train()
    for images, mask in train_loader:
        images, mask = images.to(device), mask.to(device)

        output = model(images)
        optimizer.zero_grad()

```

```

loss = loss_fn(output, mask)
loss.backward()
optimizer.step()

total_loss_UNet += loss.item()

epoch_loss_UNet = total_loss_UNet / len(train_loader)
print(f"epoch [{epoch+1}/{epochs}] , loss [{epoch_loss_UNet:.4f}]")

```

- loss = 0.0725로 안정적으로 학습된것을 확인
- 테스트 데이터셋으로 평균 loss 와 정확도 판단.

2-2) 모델 검증

```

import torch

def UNet_test(model, test_loader, loss_fn, device):
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for imgs, masks in test_loader:
            imgs = imgs.to(device)    # (B,3,H,W)
            masks = masks.to(device)  # (B,H,W)

            outputs = model(imgs)      # (B,C,H,W)
            loss = loss_fn(outputs, masks)
            total_loss += loss.item()

            preds = outputs.argmax(dim=1) # (B,H,W)
            correct += (preds == masks).sum().item()
            total += masks.numel()

    avg_loss = total_loss / len(test_loader)
    pixel_acc = correct / total

```



```
print(f"[Test] loss: {avg_loss:.4f}, pixel acc: {pixel_acc:.4f}")
```

```
UNet_test(model, test_loader, loss_fn, device)
```

- test 데이터셋으로 평가 결과
- 평균 loss 는 0.0952, 정확도는 0.9692 확인

2-3) 검증 결과 시각화

```
with torch.no_grad():
    test_iter = iter(test_loader)
    plt.figure(figsize=(15, 80))

    plot_index = 1
    for images, masks in test_iter:
        images = images.to(device)
        outputs = model(images)
        pred_masks = torch.argmax(torch.softmax(outputs, dim=1), dim=1).cpu().numpy()

        for b in range(images.size(0)):
            plt.subplot(20, 3, plot_index)
            plt.imshow(images[b].cpu().permute(1, 2, 0))
            plt.title("Original Image")
            plt.axis("off")
            plot_index += 1

            plt.subplot(20, 3, plot_index)
            plt.imshow(masks[b].cpu().numpy(), cmap="jet")
            plt.title("Ground Truth Mask")
            plt.axis("off")
            plot_index += 1

            plt.subplot(20, 3, plot_index)
            plt.imshow(pred_masks[b], cmap="jet")
            plt.title("Predicted Mask")
            plt.axis("off")
            plot_index += 1
```

```

if plot_index > 20 * 3: # 20개 이미지까지 시각화
    break
if plot_index > 20 * 3:
    break

plt.tight_layout()
plt.show()

```

